LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 14. Functions

Lecture: 156. Variable Length Keyword Arguments

Section 1: Lecture Summary

This lecture focuses on **variable length keyword arguments** in Python functions, distinguishing them from variable length positional arguments covered previously. It explains how to define a function parameter that accepts an arbitrary number of keyword arguments using **double asterisks ``, resulting in a dictionary of key-value pairs. The lecture demonstrates creating, passing, and iterating over these keyword arguments and explores rules about argument ordering when mixing positional, keyword, and variable length arguments. Special attention is given to syntax constraints such as disallowing any parameters after** `kwargs` and the behavior of arguments placed between `args` and `**kwargs`, which become keyword-only parameters. The lecture concludes by emphasizing the expressive power of Python's keyword arguments compared to other languages.

Section 2: Key Concepts and Explanations

- **Variable Length Positional Arguments (`args`)**

  - Use a single asterisk `` before a parameter name (commonly `args`) to accept an arbitrary number of positional arguments.

  - Inside the function, `args` is treated as a tuple of all positional arguments passed beyond the explicitly defined ones.

- **Variable Length Keyword Arguments (**`kwargs`)**

  - Use double asterisks `**` before a parameter name (commonly `kwargs`) to accept an arbitrary number of keyword arguments.

  - These keyword arguments are packed into a dictionary, where each key is the argument name and each value is the passed respective value.

  - Example: `def fun(**kwargs)`

- **Behavior and Usage of** `kwargs`**

  - When printing or accessing `kwargs`, it behaves like a dictionary.

- You can iterate over the dictionary items using `for item in kwargs.items():`, where each `item` is a tuple `(key, value)`.

- Conditional logic can filter for specific keys inside the loop.

- **Argument Ordering Rules**

- No parameter can follow `**kwargs` — attempting to place any parameter after variable keyword arguments causes a syntax error.

- Parameters can precede `**kwargs` and can be passed either positionally or by keyword.

- If parameters are placed **between** `args` and **`kwargs`, they become** keyword-only parameters**. These must be specified by keyword when calling the function.

- When mixing `args`, keyword-only parameters, and `**kwargs` in a function, the general order is:

1. Regular positional parameters (optional)

2. `args` (variable length positional)

3. Keyword-only parameters (after `args`)

4. `**kwargs` (variable length keyword)

- **Comparison of Data Structures Used**

- `args` collects extra positional arguments into a **tuple**.

- **`kwargs` collects extra keyword arguments into a** dictionary**.

- **Python's Strength with Keyword Arguments**

- Python's support for keyword arguments makes it more flexible and powerful compared to many other languages that only support positional variable arguments.

Section 3: Example Code and Use Cases

Example 1: Variable length keyword arguments as a dictionary

```
def fun(**kwargs):
    print(kwargs)

fun(a=5, b=10, c=15)
```

Explanation: This function collects the keyword arguments into a dictionary named `kwargs` and prints it. Output: `{'a': 5, 'b': 10, 'c': 15}`

Example 2: Iterating over kwargs dictionary and printing value for key 'b'

```
def fun(**kwargs):
    for item in kwargs.items():
        if item[0] == 'b':  # item[0] is key
            print(item[1])  # item[1] is value

fun(a=5, b=10, c=15)
```

Explanation: Iterates over the keyword arguments. Prints only the value corresponding to the key `'b'`. Output: `10`

Example 3: Positional arguments before **kwargs are allowed

```
def fun(a, b, **kwargs):
    print(a, b)
    print(kwargs)

fun(a=5, b=10, c=15)
```

Explanation: `a` and `b` are regular parameters and `kwargs` collects any extra keyword arguments. Output:

```
5 10
{'c': 15}
```

Example 4: *args, keyword-only parameters, and **kwargs together

```
def fun(*args, a, b, **kwargs):
    print(args)
    print(a, b)
    print(kwargs)

fun(1, 2, 3, a=4, b=5, x=6, y=7)
```

Explanation:

- `args` collects `(1, 2, 3)` positional arguments.

- `a` and `b` are keyword-only arguments that must be specified by keyword.

- `kwargs` collects additional keyword arguments (`x` and `y`).

Output:

```
(1, 2, 3)
4 5
{'x': 6, 'y': 7}
```

Invalid example: Cannot have arguments after **kwargs

This raises a SyntaxError: arguments cannot follow variable keyword arguments

```
def fun(**kwargs, a, b):
    print(kwargs)
```

Explanation: The syntax is invalid. Parameters cannot be declared after `**kwargs`.

Section 4: Key Takeaways

- Use `args` for variable length positional arguments; this collects arguments into a tuple.

- Use `**kwargs` for variable length keyword arguments; this collects arguments into a dictionary.

- Parameters **must not** come after `**kwargs` — it leads to a syntax error.

- Parameters can precede `**kwargs` and accept values positionally or by keyword.

- Parameters placed between `args` and `**kwargs` are keyword-only and must be passed by keyword.

- Iterating over `kwargs.items()` allows access to individual key-value pairs for processing.

- Python's support for keyword arguments enhances function flexibility beyond most other languages.

- Keep argument order clear to avoid syntax errors and unexpected behaviors in function definitions.